P3. Assignment 3.1 Contapache. Secure Configuration of Web Server -Canvas-

Contapache Secure Configuration of Web Server

Introduction

- **What is Docker?** Docker is a tool that provides developers and system administrators with an easy way to deploy their applications in a sandboxed, contained environment. These *containers* can then be run on the host operating system.
- **Why use Docker?** The key benefit of Docker is that it allows users to package an application, with all of its dependencies, into a standardized unit for deployment. Unlike virtual machines, containers do not have high overhead and, hence, enable more efficient usage of the underlying system and resources.
- **What are Containers?** The industry standard today is to use Virtual Machines (VMs) to run applications. VMs are great at providing complete isolation for applications: there are very few ways a problem in the host operating system can affect the software running in the guest operating system, and vice-versa. However, this isolation comes at great cost — the computational overhead spent in virtualizing hardware for a guest OS to use is substantial. Containers take a different approach by leveraging the low-level mechanics of the host operating system. They can thus provide most of the isolation facilities of the virtual machines at a fraction of their computing power.
- **Why use Containers?** Containers offer a logical packaging mechanism that allows applications to be abstracted from the environment in which they actually run. This decoupling allows container-based applications to be easily and consistently deployed, regardless of whether the target environment is a private datacentre, the public cloud, or even a developer's personal laptop. In turn, developers have the ability to create consistent environment that are isolated from rest of the system and can be run anywhere.

Lifecycle of Docker Containers

1. Dockerfile:

Every docker container is created from a Dockerfile. A Dockerfile is a text document that contains all of the commands a user could use on the command line to assemble an image.

Example Dockerfile:

FROM ubuntu

COPY . /app

RUN make /app

CMD python /app/app.py

Explanation: The above Dockerfile contains four commands, each creating a layer.
The FROM statement starts out by creating a layer from the ubuntu:18.04 image (Openly available). The COPY command adds some files from your Docker client's current directory. The RUN command builds your application using the make command. Finally, the last layer specifies what command to run within the container.

2. Docker Image:

A Docker image is built up from a series of layers. Each layer represents an instruction in the image's Dockerfile and except the very last one, are all read-only.

Each additional layer, that are all stacked over each other, only contains a set of differences from the ones before it. When you create a new container, you add a new writable layer, often called the "container layer," on top of the underlying layers. All changes made to the running container, such as writing new files, modifying existing files, and deleting files, are written on this thin writable container layer. The diagram below shows a container based on the Ubuntu 18.04 image.

```
_____________________
  Thin R/W Layer
_____________________
|   |   |   |   |
|   91sef2r93r3r   |
|   61rbw2473o9p   |
|   21sew2e73o9p   |
|   31grw9e69u2b   |
|   |   |   |   |
_____________________
```

---

3. Docker Container:

The major difference between a container and an image is the top writable layer. All writes to the container that add new or modify existing data are stored in this writable layer. When the container is deleted, the writable layer is also deleted, but the underlying image remains unchanged.

Because each container has its own writable container layer, and all changes are stored in this container layer, multiple containers can share access to the same underlying image but can still have their own data state.

As discussed above, each container has its own data state. By default all files created inside a container are stored on a writable container layer. This means that:

- The data doesn't persist when that container no longer exists, and it can be difficult to get the data out of the container if another process needs it.
- A container's writable layer is tightly coupled to the host machine where the container is running. You can't easily move the data somewhere else.

*Note: Writing into a container's writable layer requires a storage driver to manage the filesystem. The storage driver provides a union filesystem, using the Linux kernel. This extra abstraction reduces performance when compared to using data volumes, which writes directly to the host filesystem.

*Solution: Docker has two options for containers to store files in the host machine so that the files persist even after the container stops. You can find a description of those methods in here

Links to an external site..

The diagram below shows multiple containers sharing the same Ubuntu 18.04 image.

```
------------  ------------  ------------  ------------
| Container 1 | Container 2 | Container 3 | Container 4|
------------  ------------  ------------  ------------
  R/W Layer     R/W Layer     R/W Layer     R/W Layer
```

```
|        |        |        |
\        \       /        /
 \        \     /        /
  \        \    /        /
   \        \   /        /
    \        \ /        /
     \        \/        /
        _____________________
          Thin R/W Layer
        _____________________
        |   |   |   |   |
        |   91sef2r93r3r   |
        |   61rbw2473o9p   |
        |   21sew2e73o9p   |
        |   31grw9e69u2b   |
        |   |   |   |   |
        _____________________
```

Basic Docker Commands

- **Containers:**

1. run: first creates a writeable container layer over the specified image, and then starts it using the specified command.
    - Syntax: docker run <options> <name_of_docker_image>
        - Attributes:
            - --name: Name of the container starting.
            - --interactive , -i: Keeps STDIN open even if not attached.

        - --tty , -t: Allocates a pseudo-TTY.
        - -d, --detach: Runs container in background and print container ID.
        - -p, --publish list: Publishes a container's port(s) to the host.
        - -v, --volume: Volumes are the preferred mechanism for persisting data generated and used by Docker containers.
    - Example 1: docker run --name nginx_container -p 80:8090 -it -d nginx
        - *publishing host port 80 to container port 8090*
    - Example 2: docker run -v /dbdata --name dbstore2 nginx /bin/bash
2. cp: copies files and folders between the Docker container and a host machine.
    - Syntax : docker cp <source> <destination>
    - Example: docker cp 72ca2488b353:/foo.txt foo.txt
        - *copies foo.txt from container to host machine*
3. exec: runs a new command in a running container.
    - Attributes:
        - --interactive , -i: Keeps STDIN open even if not attached
        - --tty , -t : Allocates a pseudo-TTY
    - Syntax: docker exec -it <container_name/container_id> <shell_command>
    - Example: docker exec -it nginx_container /bin/bash
4. inspect: provides detailed information on the constructs controlled by Docker.
    - Attributes:
        - --format , -f: Formats the output using the given Go template
    - Example: docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' $INSTANCE_ID
        - *Get an instance's IP address*
5. ps: lists all the running containers by default.
    - Attributes:
        - --all , -a: Shows all containers (default shows just running)

        - --filter , -f: Filters output based on conditions indicated
    - Example : docker ps -a --filter "name=nginx" --filter "status=paused"
6. stop: stops one or more running containers.
    - Attributes:
        - --time , -t: Indicates the seconds for stop to wait before killing it; the default is 10 seconds.
    - Example: docker stop -t 30 nginx
7. kill: kills the container by stopping its execution *immediately*. The difference between 'docker kill' and 'docker stop' is that 'docker stop' gives the container time to shutdown gracefully. In situations when stopping a container is taking too much time, one can opt to kill it.
    - Syntax: docker kill <container_id/container_name>
    - Example: docker kill nginx
8. rm: removes one or more containers.
    - Attributes:
        - --force , -f: Forces the removal of a running container (uses SIGKILL)
    - Syntax: docker rm <container_id/container_name>
    - Example:
        - docker rm nginx
        - docker rm $(docker ps -a -q)
            - *deletes all containers*
9. restart: restarts one or more containers in case that a container does not respond or function as it should.
    - Syntax: docker restart <container_id/container_name>
    - Example: docker restart nginx
10. version: provides information on the version of docker engine
    - Example: `docker –version

- **Repository:**

1. login: logins the user into a docker repository like Docker hub. Enter username and password for docker hub when prompted.
    - Example: docker login
2. commit: creates a new image from a container's changes.
    - Syntax: docker commit -it <container_name/container_id> <docker_image_name>:<tag>
    - Example: docker commit c3f279d17e0a cannedcyberlabs/nginx:version3
3. pull: pulls a docker image from a docker repository.
    - Syntax: docker pull <name_of_docker_image>
    - Example: docker pull nginx
4. push: pushes an image to a registry.
    - Syntax: docker push <name_of_registry>:<name_of_docker_image>
    - Example: docker push cannedcyberlabs:nginx

- **Images:**

1. image: this command deals with and performs various operations docker images.
    - Subcommands:
        - *list*: lists all docker image on the host systems.
        - *build*: builds a docker image from Dockerfile.

Create your Docker Hub account

The global repository for docker images is called **Docker Hub** https://hub.docker.com/Links to an external site.. There, you can find a lot of images that you can pull and run in your own system. You can also push images to the repository and into your account, which can be useful for sharing and to have them available for you from other systems.

To create your account you can sign up through this link: Docker Hub Sign-upLinks to an external site.. > Remember that when you make changes in a container, those changes will be lost if you do not *commit* them and create a new image. You then need to *push* them into the repository to make them available.

Deliverables

This assignment requires a report—submitted via Canvas—that includes both screenshots and explanations. The report must be organized according to the same structure as the assignment. For each screenshot or explanation, clearly indicate which part and task of the assignment it corresponds to.

As part of your submission, you must also include the full docker pull command that can be used to retrieve your final Docker image after completing all tasks. **Include this command at the beginning of your report.**

Part 0: Setup Docker in your system

- Make sure that you have docker installed in your Linux system
    - If you don't have it, go ahead and install it (apt-get install docker docker.io -y). Check this linkLinks to an external site. if you have doubts about how to do it.
- Verify that your system is ready, by *pulling* and *running* the hello-world docker container.

$ sudo docker run hello-world

[sudo] password for test:

Unable to find image 'hello-world:latest' locally

latest: Pulling from library/hello-world

0e03bdcc26d7: Pull complete

Digest: sha256:6a65f928fb91fcfbc963f7aa6d57c8eeb426ad9a20c7ee045538ef34847f44f1

Status: Downloaded newer image for hello-world:latest

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:

https://hub.docker.com/

For more examples and ideas, visit:

https://docs.docker.com/get-started/

test@linuxHO:~$

Part 1: Run apache server into the container and map it to a host machine port

- Pull and run the httpd docker image in your system.

$ sudo docker pull httpd

latest: Pulling from library/httpd

afb6ec6fdc1c: Pull complete

5a6b409207a3: Pull complete

41e5e22239e2: Pull complete

9829f70a6a6b: Pull complete

3cd774fea202: Pull complete

Digest: sha256:db9c3bca36edb5d961d70f83b13e65e552641e00a7eb80bf435cbe9912afcb1f

Status: Downloaded newer image for httpd:latest

verify that the image is downloaded in your system and it is listed.

$sudo docker image ls

REPOSITORY TAG IMAGE ID CREATED SIZE

httpd latest d4e60c8eb27a 6 days ago 166MB

hello-world latest bf756fb1ae65 4 months ago 13.3kB

test@linuxHO:~$

- Run the httpd image in your system and map linux local port 8090 with the 80 port in the container (which we have named 'contapache').

$ sudo docker run -dit --name contapache -p 8090:80 httpd

def417356a975b77433df4737e8f973d3918171883c15bb306418f59ded7b47c

- Then, you can check that the container is running in the background with the pscommand.

$ sudo docker ps

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

def417356a97 httpd "httpd-foreground" 8 minutes ago Up 8 minutes 0.0.0.0:8090->80/tcp contapache

- Now, connect to the container web server using a terminal web browser like curl and your localport 8090.

$ curl localhost:8090

<html><body><h1>It works!</h1></body></html>

--> Submit screenshot-1 showing that you were able to successfully connect to the container webserver.

- Modify the default 'index.html' file in the container (located at /usr/local/apache2/htdocs/index.html) to a new one like this:

```
<html>

<header><title>CY5001 Assignment </title></header>

<body>

I am ..{{YOUR FULL NAME}}... and this is my CY5001 assignment on  ..{{CURRENT DATE}}..

</body>

</html>
```

- To complete this task, you can create the index file in your Linux system and then cp it into the docker container. Otherwise, execute exec a bash terminal in the container and create the file directly in it. In any case, once you have finish, when you connect back to the container web server the result should be something link this:

```
$ curl localhost:8090
<html>
<header><title>CY5001 Assignment </title></header>
<body>
I am ..{{YOUR FULL NAME}}... and this is my CY5001 assignment on  ..{{CURRENT DATE}}..
</body>
</html>
```

--> Submit screenshot-2 showing that you were able to successfully connect to the container webserver and obtain the new 'index.html' file.

- Now before moving forward, let's save the image of the container so you preserver this image. To do this you need to **commit** your container to a new image before you stop the container. You should be able to see your image created in your system.
- $ sudo docker image ls
- REPOSITORY TAG IMAGE ID CREATED SIZE
- sierraneu/contapache 1.0 ce2665823e8f About a minute ago 166MB
- httpd latest d4e60c8eb27a 6 days ago 166MB

hello-world latest bf756fb1ae65 4 months ago 13.3kB

Part 3: Start a secure Apache HTTPS in the container and Map it

- Connection going over the HTTP (the normal web protocol) is unencrypted, so all content exchanged can be access by any communication intermediation party. There is a protocol intended to prevent this vulnerability, which is the HTTPS. This part of the assignment has the objective to deploy a HTTPS server into the same container that you are running the previous HTTP service (both services HTTP and HTTPS will be running together but in different TCP Ports -80 and 443-) .
- The first step in order to provide a HTTPS server is to create a self-sign certificate with openssl. There are plenty of sources that can tell you how to do it. Once you figure it out, make sure that you provide this information as your server subject:

subject: C= US; ST= MA; L= NEU; O= MSCY; CN= userid.cy5001.com

'userid' in the CN being your NEU user ID.

- Make sure that you store the private key and the certificate files of the server in the right location. You need to provide that location to configure the HTTPS server.
- Then, you need to install the openssl module into your container (apt-get install -y openssl) and enable SSL and provide the proper configuration into the ***httpd-ssl.conf file.*** The following snippet shows a regular configuration file that you need to adapt to your system.

```
LoadModule ssl_module modules/mod_ssl.so
Listen 443
<VirtualHost _default_:443>
DocumentRoot "/usr/local/apache2/htdocs"
ServerName www.example.com
SSLEngine on
SSLCertificateFile "/usr/local/apache2/conf/ssl/apache.crt"
SSLCertificateKeyFile "/usr/local/apache2/conf/ssl/apache.key"
<FilesMatch "\.(cgi|shtml|phtml|php)$">
  SSLOptions +StdEnvVars
</FilesMatch>
<Directory "/usr/local/apache2/cgi-bin">
  SSLOptions +StdEnvVars
```

</Directory>
</VirtualHost>

- The final step is to Include the SSL configuration you just created, into the main *httpd.conf* file.
- Let's save the image of the Part 3. To do this you need to **commit** your container to a new image before you stop the container. Use a different TAG for this new image, so you preserve the image you did for Part 2.

Part 4:

- Once you have a new image you can stop the container.
- Now, run it again and make sure that you map both ports of the container as:
    - HTTP --> 80
    - and HTTPS --> 443
- Before pushing your new image to your docker repository, make sure that your Part3 image correctly maps the container ports (80, 443) to whatever ports you want in your linux host.
- The last step is to push the new image with the changes you made into the Docker Hub repository using your Docker Hub account.
- To upload/push your image, you need to docker login in your account from your linux system, then docker commit the working version of the image and finally docker push.

Deliverable (Canvas)

Submit a PDF report that includes all required screenshots. Each screenshot must be clearly labeled to indicate the corresponding part of the assignment. Points will be deducted for missing or unclear labels.

At the top of your PDF report, include the full docker pull command needed to retrieve your image from your repository (*e.g.: docker pull sierraneu/vulpass:T49*). This command must specify the complete image name, including the correct version and tag (if applicable).

Points will be deducted if the image name is missing, incomplete, or the version/tag is not explicitly included.

Grading

Please see here the rubric for the assignment.

Concept	Points
Part 1: Successful ‘screenshot-1’	10
Part 2: Successful ‘screenshot-2’	30
Part 3: Create new image of modified container	30
Part 4: Successfully pushing the image of your container to the docker hub repository. When your container image is pulled from docker hub, it needs to include the changes you have made.	30

References

- https://www.freecodecamp.org/news/docker-simplified-96639a35ff36Links to an external site.